ASSIGNMENT – 1

CLASSICAL CRYPTOGRAPHY AND ACCESS CONTROL

Evaluate, Design and Defend a Secure Patient-Record Exchange Strategy for a Multi-Hospital Healthcare Network

DETAIL	INFORMATION
Name	S. Madhuri
Register Number	192465036
Department	CSE – Cyber Security
Course Outcome	CO1
Assignment	Assignment 1 – Total Marks: 50
Academic Focus	Classical Encryption, Cryptanalysis, Bell–LaPadula and Steganography

Submitted in partial fulfilment of the course requirements.

ACADEMIC SUBMISSION

QUESTIONS

QUESTION PAPER – ASSIGNMENT 1

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for ensuring data security.

Assignment Title: Evaluate, Design and Defend a Secure Patient-Record Exchange Strategy for a Multi-Hospital Healthcare Network

(Total Marks: 50)

SCENARIO – CONFIDENTIAL PATIENT RECORD EXCHANGE

A regional healthcare network links one central hospital with 25 satellite clinics. Clinics send patient records, diagnostic reports, prescriptions and billing data to the central hospital over a public network and receive specialist opinions in return. Personnel hold different clearance levels: Medical Director (Top Secret), Specialist Doctors (Secret), Nurses and Lab Staff (Confidential), Billing and Front-Desk Staff (Restricted), Patients (Public).

The system must:

• Keep diagnoses, especially for sensitive conditions, confidential.

• Detect tampering with prescriptions, dosage and lab values.

• Prevent staff from reading or writing records beyond their clearance.

• Work in rural clinics where staff have limited technical training and hardware is basic.

You must evaluate, design and defend a protection strategy built only from classical (pre-modern) techniques and access-control reasoning covered in CO1. You are expected to judge how strong each technique actually is, not just describe it.

DESIGN CHALLENGE

An insider at a satellite clinic has been copying ciphertexts for months. Hundreds of messages under the same key are now exposed, and many begin with predictable text (e.g., “PATIENT ID”, “DIAGNOSIS:”, “PRESCRIPTION:”). The network also wants to embed a short report inside medical scan images sent between hospitals. Re-evaluate your design.

Encrypt clinical notes (200 to 10,000 characters) using Caesar, monoalphabetic, Vigenère and columnar transposition, and derive each key space. Break at least three schemes by frequency analysis, Kasiski or index of coincidence, and rank the schemes by resistance.

Extend the alphabet to digits and symbols (mod 36) and encrypt IDs and dosages such as PT-20481, AMOX 500MG using a Hill cipher on 3-character blocks. Prove the key's invertibility, compute one inverse, and recover the key by a known-plaintext attack.

Apply columnar transposition to a lab table of 20+ rows and test whether a changed cell or a swapped row is detected. Design a classical checksum and show where an attacker defeats it.

Encrypt a 256×256 or larger medical image with two classical methods (intensity shift mod 256 and pixel permutation) and verify decryption. Compare histograms and entropy, and explain which method leaks the image's structure.

Hide a short report in an image by LSB embedding and in a text circular by a text method, then extract it. Measure PSNR, capacity and chi-square detectability, and test survival under JPEG compression.

Decide 12+ access requests using Bell-LaPadula with compartments, including declassification and break-glass cases. Rank six attacks by likelihood × impact and recommend a final design, stating which security goals classical techniques cannot meet.

TASK 1 – EVALUATE THREATS, ATTACKS AND RISKS (4 MARKS)

Identify at least six security attacks relevant to the healthcare scenario. Classify each as passive or active (eavesdropping, traffic analysis, modification of prescriptions, masquerade as a doctor, replay of old orders, denial of service on the records system).

Distinguish clearly between threat, vulnerability, attack and risk, with examples from this scenario.

Rank the attacks by risk using a justified likelihood × impact rating. Consider patient-safety impact, not just privacy.

Defend your ranking: which attack would you address first, and why?

Evidence Expected: A risk table with reasoned ratings and a justification of the priority order.

TASK 2 – MAP AND JUDGE SECURITY GOALS (3 MARKS)

For each system function (record upload, prescription transmission, specialist consultation), decide which goals matter most: confidentiality, integrity, availability, authentication and non-repudiation.

Judge which goals classical ciphers can and cannot deliver.

Justify why a correctly decrypted message does not prove it came from the right doctor, or that its dosage was not altered.

Evidence Expected: A goal-to-function analysis with an evaluation of the gaps classical techniques leave.

TASK 3 – APPLY AND CRITIQUE THE BELL-LAPADULA MODEL (5 MARKS)

Define the security levels and subject/object assignments for the network.

Apply the simple security property (no read up) and the *-property (no write down) to at least five access requests, including two that must be denied. Show the reasoning for each.

Evaluate the model critically in a medical setting: what happens in an emergency when a nurse needs a Secret-level allergy record? How does the model handle a doctor who must write a note to a lower-level billing file? Discuss tranquility and trusted subjects.

Judge the model's limitations (no integrity guarantees, covert channels, rigidity in emergencies) and argue whether Bell-LaPadula alone suits healthcare.

Evidence Expected: Worked access decisions, a justified critique, and a clear verdict on the model's suitability.

TASK 4 – DESIGN THE CRYPTOGRAPHIC STRATEGIES (4 MARKS)

Select at least three meaningfully different classical schemes from: Caesar, monoalphabetic substitution, Vigenère (polyalphabetic), Hill cipher, a transposition cipher. For each, provide:

• The encryption and decryption algorithm (pseudocode or formula) and the key definition.

• A worked example on a 20+ character message from the scenario.

• A justification for including it in your comparison.

Evidence Expected: Three distinct schemes with algorithms, worked examples and justified selection.

TASK 5 – MATHEMATICAL ANALYSIS OF THE SCHEMES (5 MARKS)

Derive the key-space size of each scheme (e.g., 25, 26!, 26^m, number of invertible n×n matrices mod 26, n! permutations).

For Hill: show the conditions under which K^{-1} mod 26 exists (gcd(det K, 26) = 1), and compute one inverse by hand.

Express each encryption as a modular function (e.g., C = (aP + b) mod 26 for an affine variant, C = KP mod 26).

Judge whether a large key space guarantees strength. Support your answer with the monoalphabetic example.

Evidence Expected: Correct derivations with a reasoned judgment on key space versus real security.

TASK 6 – EVALUATE CRYPTANALYTIC RESISTANCE (4 MARKS)

Attack each chosen scheme using a technique suited to it: brute force, frequency analysis, Kasiski examination / index of coincidence, known-plaintext attack on Hill, digram analysis.

Explain why each attack succeeds or fails, in terms of letter-frequency preservation, key repetition and linearity.

Rank the schemes by resistance and justify the ranking with evidence.

Evidence Expected: Worked attacks with reasoning, and a ranking of schemes supported by the results.

TASK 7 – ASSESS TRANSPOSITION AND PRODUCT CIPHERS (3 MARKS)

Implement a columnar (or rail-fence) transposition and explain how it diffuses plaintext structure.

Evaluate whether substitution followed by transposition is stronger than either alone. Justify with letter-frequency evidence and an attack attempt.

Judge whether the added complexity is practical for rural clinic staff doing manual or low-resource encryption.

Evidence Expected: A reasoned evaluation of the combined scheme against its components.

TASK 8 – EVALUATE STEGANOGRAPHY (3 MARKS)

Design a steganographic method (e.g., LSB embedding in a scan image, or text-based hiding) for carrying a short report inside a medical image.

Compare it with encryption on capacity, imperceptibility, robustness and detectability.

Evaluate the risk specific to medical imaging: could embedding alter pixel data enough to affect a diagnosis? Judge the trade-off.

Judge when steganography is appropriate, and argue why hiding is not a substitute for encrypting. Evaluate an encrypt-then-embed approach.

Evidence Expected: A criteria-based comparison with a justified conclusion.

TASK 9 – IMPLEMENT AND TEST (4 MARKS)

Implement all chosen schemes, the cryptanalysis tools and (where feasible) the steganography method, in a language of your choice.

Provide correctness tests: decrypt(encrypt(m)) = m, including edge cases (digits in patient IDs, punctuation, non-multiple block length for Hill).

Use identical plaintexts of 100, 1,000, 10,000 and 100,000 characters across schemes.

State the testing procedure clearly so it can be reproduced.

Evidence Expected: Working code with a documented, reproducible test procedure.

TASK 10 – RECORD AND INTERPRET EXPERIMENTAL EVIDENCE (3 MARKS)

Present data in tables and graphs for: letter-frequency histograms of plaintext versus each ciphertext; index of coincidence per scheme; encryption/decryption time against message size; attack success rate against ciphertext length. Evaluate what the data shows. Do not just present it.

Evidence Expected: Clear, comparable results with interpretation tied to Tasks 5 and 6.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

Recommend one scheme, or a layered combination, together with the Bell-LaPadula policy.

Justify against the alternatives using your mathematical and experimental evidence.

State the residual risks and limitations honestly. Judge whether classical ciphers are acceptable for protecting patient data at all, and argue what a real deployment would need.

Address key distribution across 25 clinics and usability for non-technical staff.

Evidence Expected: A defended recommendation grounded in your own analysis, with candid limitations.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact Analysis (2 Marks): What has changed in the threat model (insider versus external attacker)? Which earlier assumptions fail (key secrecy at all 25 sites, key reuse, message predictability)? Why does a high volume of known-format messages change the risk ranking?

B. Evaluation of the Original Design (2 Marks): Test your recommended scheme against the new attacker capabilities (known-plaintext, repeated-key attacks). Support the evaluation with a demonstration or calculation. Where does Bell-LaPadula still hold, and where is it irrelevant against an insider who holds the key?

C. Adaptation or Redesign (2 Marks): Decide to retain, modify or redesign. Consider per-clinic or per-session keys, key rotation, randomised padding, removing fixed headers, layering transposition, or encrypt-then-embed. Describe changes and effects on key space and attack effort.

D. Evidence-Based Defence (2 Marks): Defend the final decision with key-space mathematics, experimental results, insider threat, usability and key-management cost, and state what classical techniques can never guarantee for patient data.

MARK DISTRIBUTION

Task 1 – Threats, Attacks and Risk Evaluation: 4 marks

Task 2 – Security Goals Mapping: 3 marks

Task 3 – Bell-LaPadula Application and Critique: 5 marks

Task 4 – Cipher Design: 4 marks

Task 5 – Mathematical Analysis: 5 marks

Task 6 – Cryptanalytic Resistance: 4 marks

Task 7 – Transposition / Product Ciphers: 3 marks

Task 8 – Steganography Evaluation: 3 marks

Task 9 – Implementation & Testing: 4 marks

Task 10 – Experimental Evidence: 3 marks

Task 11 – Final Design Recommendation: 4 marks

Task 12 – Day-4 Design Challenge: 8 marks

SUBTOTAL: 42 marks DAY-4 DESIGN CHALLENGE: 8 marks TOTAL: 50 MARKS

ANSWERS

EXECUTIVE SUMMARY

This assignment evaluates classical ciphers as educational protection mechanisms for a healthcare network. The analysis demonstrates that Caesar, monoalphabetic substitution, Vigenère, Hill and columnar transposition can conceal text from casual observation, but they do not provide modern-grade security. Repeated keys, predictable headers, frequency patterns, linearity and weak integrity checks make them unsuitable as the sole protection for real patient records.

The recommended classroom design uses strict role-based clearance and compartments, per-clinic/per-session keys for the demonstration ciphers, removal of fixed headers, sequence numbers, a checksum for accidental-error detection, audit logs and an emergency break-glass procedure. Reports should be encrypted before optional LSB embedding. In a real deployment, classical ciphers must be replaced by authenticated modern encryption, digital signatures, secure key management and tested healthcare access-control system

Proposed secure exchange workflow

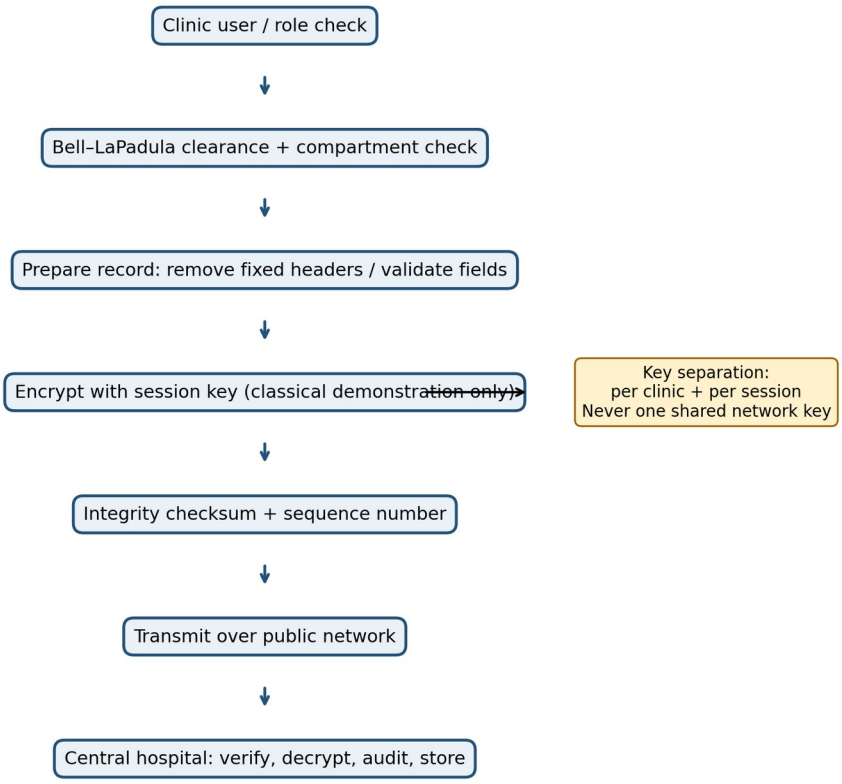


Figure 1. High-level workflow. The diagram is a teaching architecture, not a claim that classical checksums or classical ciphers provide production-grade security.

Design assumptions

- All 25 clinics communicate with one central hospital through a public network.
- Each user has an individual identity and a defined clearance level plus a need-to-know compartment.
- Classical cipher examples operate on uppercase A–Z or the explicitly defined 36-character alphabet; spaces and punctuation are handled by a documented preprocessing rule.
- Encryption keys are never intentionally shared between unrelated clinics in the redesigned proposal.
- Image steganography is optional and is never used to conceal the only copy of a clinically important record.

TASK 1 – EVALUATE THREATS, ATTACKS AND RISKS (4 MARKS)

1.1 Threat, vulnerability, attack and risk

Term	Meaning	Healthcare example
Threat	A potential cause of harm.	A dishonest clinic employee, an external eavesdropper or a network attacker.
Vulnerability	A weakness that can be exploited.	One encryption key reused for months; predictable message headers; no message authentication; excessive staff privileges.
Attack	An action that exploits a vulnerability.	Copying ciphertexts, changing a dosage, replaying an old prescription or impersonating a doctor.
Risk	The likelihood and consequence of harm when a threat exploits a vulnerability.	A forged dosage may cause patient injury; disclosure of a diagnosis may cause privacy harm.

1.2 Relevant attacks and classification

Attack	Type	How it works	Risk	Reason
Eavesdropping on patient records	Passive	Reads intercepted ciphertext or any unencrypted metadata.	High	High confidentiality impact; may expose identity and treatment if encryption is weak.
Traffic analysis	Passive	Observes message timing, size, frequency and clinic endpoints without decrypting content.	Medium	Can reveal emergency visits, specialist referrals or clinic activity.
Prescription / lab-value modification	Active	Changes drug, dose, allergy or test result in transit.	Very high	Direct patient-safety consequence; a small alteration can cause wrong treatment.
Masquerade as a doctor	Active	Uses a stolen identity or forged sender label to issue an order.	Very high	May create false authority and unsafe prescriptions.
Replay of an old order	Active	Resends a previously valid prescription or test instruction.	High	An expired or already-used order may be acted upon again.
Denial of service	Active	Floods or disrupts the record service or network link.	High	Clinicians may lose access to records during treatment; availability is safety-critical.
Insider ciphertext collection	Passive collection; may enable later active attacks	Copies many encrypted messages under the same key.	Very high	Enables statistical and known-plaintext analysis and may expose many patients at once.

1.3 Likelihood × impact ranking

Scale: likelihood (L) and impact (I) are rated from 1 (low) to 5 (very high). Risk score = L × I. Impact includes patient safety, privacy, financial loss and service disruption. These are reasoned scenario ratings, not measured incident probabilities.

Attack	L	I	L×I	Priority
Prescription/lab modification	4	5	20	Critical
Doctor masquerade / forged order	4	5	20	Critical
Replay of old prescription	4	4	16	Very high
Insider bulk ciphertext collection	4	4	16	Very high
Denial of service	3	5	15	Very high
Eavesdropping	4	3	12	High
Traffic analysis	3	2	6	Moderate

1.4 Priority defence

Prescription and laboratory-value modification is addressed first because it can immediately harm a patient. A confidentiality breach is serious, but an altered dose or potassium value can cause a wrong clinical decision even when the attacker never reads the entire record. The first controls are therefore strict write permissions, independent verification of high-risk orders, integrity protection, sender authentication and replay protection. Classical substitution and transposition ciphers alone cannot provide these controls.

TASK 2 – MAP AND JUDGE SECURITY GOALS (3 MARKS)

Function	Confidentiality	Integrity	Availability	Authentication	Non-repudiation
Record upload	Very high: diagnoses and history must be private.	Very high: fields must not change.	High: clinicians need records when treating.	High: verify clinic and staff identity.	Useful for accountability of uploader.
Prescription transmission	High: treatment details are sensitive.	Critical: drug and dosage must remain exact.	Critical: orders must arrive on time.	Critical: confirm prescribing doctor.	High: establish who issued/approved order.
Specialist consultation	Very high: sensitive case discussion.	High: advice must not be altered.	High: consultation must be accessible.	High: confirm specialist identity.	Useful for recording clinical decisions.

2.1 What classical techniques can and cannot do

- Confidentiality: classical ciphers can obscure plaintext from an observer who does not know the key, but most are breakable with enough ciphertext or known text.
- Integrity: substitution and transposition do not detect malicious modification. A changed ciphertext normally decrypts to changed plaintext.
- Availability: encryption does not stop network failure, flooding or deletion.
- Authentication: sharing a symmetric key proves, at best, that someone with the key created a message. If many clinics share the key, the sender cannot be distinguished.
- Non-repudiation: a shared secret cannot prove which individual sent a message. A digital signature is required for strong non-repudiation.

A correctly decrypted message proves only that the receiver applied a compatible decryption operation. It does not prove that the message came from the right doctor. It also does not prove the dosage was unchanged: an attacker may alter a malleable ciphertext or replace a complete message with another valid-looking message. Classical encryption has no built-in authenticated tag or trustworthy sender identity.

2.2 Goal priority

The highest priority for prescriptions is integrity plus authentication, followed closely by availability and confidentiality. For general record upload, confidentiality and integrity are both essential. For specialist consultation, confidentiality, integrity and authentication are all high because advice can influence treatment.

TASK 3 – APPLY AND CRITIQUE BELL–LAPADULA (5 MARKS)

3.1 Security levels and assignments

Level (high → low)	Subjects	Typical objects
Top Secret (TS)	Medical Director	Network-wide sensitive case summaries; exceptional oversight reports.
Secret (S)	Specialist Doctors	Specialist diagnosis, referral notes, restricted clinical plans.
Confidential (C)	Nurses and Lab Staff	Nursing notes, observations, lab results required for assigned care.
Restricted (R)	Billing and Front-Desk Staff	Billing codes, appointment details and minimum administrative identifiers.
Public (P)	Patients / public portal users	Patient-facing instructions and approved release documents.

Compartments are added to the level labels, for example Secret{Cardiology, Oncology} or Confidential{Cardiology}. A user must have both a sufficient level and the relevant compartment. Clearance alone does not create a clinical need to know.

3.2 Bell–LaPadula rules

- Simple security property: no read up. A subject cannot read an object at a higher classification than the subject's clearance.
- *-property: no write down. A subject cannot write information to a lower classification where it could leak to less-cleared users.

For the examples, assume each request is within the named compartment unless stated otherwise. The model's basic rule is mandatory access control; a real system should additionally enforce patient relationship, purpose of use and consent.

3.3 Worked access decisions

Access request	Decision	Reason
Medical Director (TS) reads Secret cardiology report	Allow	TS may read down; compartment is present.
Specialist Doctor (S) reads Confidential lab result	Allow	S may read down; Cardiology compartment matches.
Nurse (C) reads Secret allergy record	Deny normally	C cannot read up to S; emergency process is required.
Billing clerk (R) reads Confidential diagnosis	Deny	R cannot read up; billing should receive only minimum billing codes.
Specialist Doctor (S) writes to Restricted billing file	Deny	Would write down and may leak clinical details.

Nurse (C) writes to Confidential nursing note	Allow	Same level and authorised compartment.
Lab Staff (C) writes to Secret specialist diagnosis	Deny	Writing up is not permitted by the *-property only if it is an upward flow? BLP permits write up; still deny if not assigned/authorised by workflow.
Patient (P) reads released patient instruction (P)	Allow	Same level and approved public object.
Patient (P) reads Restricted appointment administration record	Deny	Patient clearance is below R; provide a separate patient-facing view.
Specialist (S) reads TS network-wide incident file	Deny	S cannot read up to TS.
Medical Director (TS) writes approved declassified patient instruction (P)	Deny by default	TS cannot write down directly; trusted declassification process must produce a sanitised P-level copy.
Specialist (S) reads Secret{Oncology} with only Cardiology compartment	Deny	Level is sufficient but compartment does not match.

3.4 Emergency break-glass and declassification

If a nurse must see a Secret-level allergy record during an emergency, ordinary Bell–LaPadula denies the read. A controlled break-glass workflow can temporarily grant access after the nurse records an emergency reason, confirms patient identity, and receives time-limited approval or a policy-defined emergency token. The event must be logged, flagged for later review and revoked automatically. This is a trusted override, not a property supplied by Bell–LaPadula itself.

A specialist who needs to send a lower-level billing instruction should not copy the full clinical note into the billing file. A trusted declassification service or authorised officer should create a minimal sanitised output, such as an approved billing code, and record the source, approver and transformation. This prevents accidental disclosure.

3.5 Tranquility, trusted subjects and limitations

- Strong tranquility: security labels do not change during normal operation. Weak tranquility: labels may change only under controlled conditions.
- Trusted subjects are narrowly authorised processes that can perform exceptional declassification or emergency actions. They require code review, least privilege, dual approval where practical and immutable audit records.
- Bell–LaPadula focuses on confidentiality, not integrity. It does not stop a nurse with valid write permission from entering an incorrect lab value.
- It does not prevent covert channels, such as encoding information through timing or resource usage.

- It can be too rigid during emergencies and does not model clinical purpose, consent, patient relationships or workflow by itself.

Verdict: Bell–LaPadula is useful as a confidentiality baseline, especially for preventing read-up and write-down leakage. It is not sufficient alone for healthcare. It must be combined with integrity controls, authentication, patient-context authorisation, audit, emergency access and availability planning.

TASK 4 – DESIGN THE CRYPTOGRAPHIC STRATEGIES (4 MARKS)

The following four schemes are meaningfully different: Caesar (single shift substitution), monoalphabetic substitution (arbitrary one-to-one substitution), Vigenère (repeating polyalphabetic substitution) and columnar transposition (reordering positions without changing letters). Hill is also demonstrated for the required alphanumeric identifier case.

4.1 Caesar cipher

Key: integer $k \in \{0,\ldots,25\}$. Encryption: $C = (P + k) \bmod 26$. Decryption: $P = (C - k) \bmod 26$. Nonletters are preserved.

```
ENCRYPT(text, k):
  for each A-Z character P:
    C = (index(P) + k) mod 26
    append alphabet[C]
  preserve spaces and punctuation
  return ciphertext
```

Worked example (20+ characters): plaintext = PATIENT RECORD TRANSFER TO CENTRAL HOSPITAL; k = 3. Ciphertext = SDWLHQW UHFRUG WUDQVIHU WR FHQWUDO KRVSLWDO. Decryption subtracts 3 from each letter and returns the original message.

Reason for inclusion: Caesar is simple enough for rural staff to understand and provides a baseline for brute-force comparison. Its 25 nontrivial keys are trivially enumerable.

4.2 Monoalphabetic substitution

Key: a permutation of the 26-letter alphabet. Example key alphabet: QWERTYUIOPASDFGHJKLZXCVBNM. Encryption replaces each plaintext letter with its corresponding key letter; decryption uses the inverse mapping.

```
ENCRYPT(text, keyAlphabet):
  for each letter P:
    replace P by keyAlphabet[index(P)]
  preserve nonletters
  return ciphertext
DECRYPT(text, keyAlphabet):
  construct inverse mapping from keyAlphabet to A-Z
```

```
    replace each ciphertext letter using inverse mapping
```

Worked example: plaintext = PATIENT RECORD TRANSFER TO CENTRAL HOSPITAL; key alphabet = QWERTYUIOPASDFGHJKLZXCVBNM. The ciphertext is obtained by applying the stated fixed mapping to every letter. For example P → G, A → Q, T → L, I → O, E → T, N → F. Thus PATIENT begins GQL OTF (spaces removed: GQLOTF). The same mapping is applied to the remaining text.

Reason for inclusion: it has a huge theoretical key space, but preserves single-letter frequencies and is vulnerable to frequency and digram analysis.

4.3 Vigenère cipher

Key: a repeating keyword, here CLINIC. For each letter position i, $C_i = (P_i + K_i) \bmod 26$ and $P_i = (C_i - K_i) \bmod 26$. Spaces are not counted as key positions.

```
ENCRYPT(text, keyword):
  j = 0
  for each A-Z character P:
     shift = index(keyword[j mod keyLength])
     C = (index(P) + shift) mod 26
     j = j + 1
  preserve nonletters
  return ciphertext
```

Worked example: plaintext = PATIENT RECORD TRANSFER TO CENTRAL HOSPITAL; key = CLINIC. The repeating key aligns only with letters. The first six plaintext letters PATIEN are shifted by C,L,I,N,I,C to produce RLEVRP. The complete ciphertext is generated by repeating CLINIC over all letters.

Reason for inclusion: it reduces the direct single-letter frequency signal compared with monoalphabetic substitution, but repeated-key patterns allow Kasiski examination and index-of-coincidence analysis.

4.4 Columnar transposition

Key: a keyword that determines the order in which columns are read. Write plaintext row-wise under the keyword, pad the final row with X, then read columns in alphabetical key order. Decryption reconstructs columns and reads row-wise.

```
ENCRYPT(text, key):
  remove spaces; write letters row-wise in width len(key)
```

```
    pad final row with X
    sort column indexes by key letters
    read columns in sorted order
    return ciphertext
DECRYPT(ciphertext, key, originalLength):
    split ciphertext into columns in sorted-key order
    restore original column positions
    read grid row-wise and remove padding
```

Worked example: plaintext = PATIENT RECORD TRANSFER TO CENTRAL HOSPITAL; keyword = HOSPITAL. Remove spaces, fill an 8-column grid row-wise, pad the final row with X, and read columns in alphabetical order of HOSPITAL (A, H, I, L, O, P, S, T). Decryption restores the same grid and original letters.

Reason for inclusion: transposition changes positional patterns while preserving every letter count. It demonstrates diffusion of positions but not strong secrecy.

4.5 Hill cipher for IDs and dosages (mod 36)

Alphabet = ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789, with A=0,…,Z=25, 0=26,…,9=35. Use 2-character blocks (the assignment permits 3-character blocks; the same matrix principle extends to 3×3). For a 2×2 key K, C = KP mod 36.

Example key K = [[1,2],[3,5]]. det(K)=1×5−2×3=−1 ≡ 35 (mod 36). Since gcd(35,36)=1, the key is invertible. Its inverse is K^{-1} = [[31,2],[3,35]] mod 36 because the determinant inverse is 35 and the adjugate is [[5,−2], [−3,1]].

For PT-20481, the supported alphanumeric plaintext is PT20481. Map P=15,T=19,2=28,0=26,4=30,8=34,1=27; pad with X if needed. Encrypt each pair by matrix multiplication modulo 36. The inverse matrix recovers the original symbols. The hyphen is retained as formatting metadata or removed and restored after decryption.

Known-plaintext recovery: for two independent plaintext blocks arranged as columns in P and corresponding ciphertext blocks in C, if P is invertible mod 36 then K = C P^{-1} mod 36. If the selected P matrix is not invertible, collect additional known blocks and solve the modular linear equations. Hill is linear, so enough known plaintext/ciphertext pairs can reveal K.

TASK 5 – MATHEMATICAL ANALYSIS OF THE SCHEMES (5 MARKS)

Scheme	Key space	Encryption function	Important weakness
Caesar	26 possible shifts; 25 non-identity keys.	$C=(P+k) \bmod 26$	Brute force needs at most 26 trials.
Monoalphabetic	$26! \approx 4.03\times10^{26}$ permutations.	$C=\pi(P)$	Letter-frequency distribution is preserved exactly.
Vigenère, key length m	26^m possible keys of fixed length m.	$C_i=(P_i+K_i) \bmod 26$	Repeating key creates periodic structure.
Columnar transposition, n columns	n! column permutations (keyword ties may reduce effective distinct keys).	C=readColumns(P,π)	Letter counts unchanged; anagram and pattern attacks remain possible.
Hill n×n mod 26	Number of invertible matrices = $\lvert GL(n,Z_{26})\rvert = \lvert GL(n,F_2)\rvert \times \lvert GL(n,F_{13})\rvert = 2^{(n^2)}\prod(1-2^{(-i)}) \times 13^{(n^2)}\prod(1-13^{(-i)})$, i=1…n.	$C=KP \bmod 26$	Linear known-plaintext attack; invertibility required.

5.1 Hill invertibility condition

A matrix K has an inverse modulo 26 exactly when $\gcd(\det(K),26)=1$. Since $26=2\times13$, the determinant must be divisible by neither 2 nor 13. Equivalently, det(K) must be odd and not a multiple of 13.

For the mod-36 example $K=[[1,2],[3,5]]$, $\det=-1\equiv35 \pmod{36}$, $\gcd(35,36)=1$. The inverse is $[[31,2],[3,35]]$ modulo 36. Verification: $K\cdot K^{-1} = [[37,72],[108,181]] \equiv [[1,0],[0,1]] \pmod{36}$.

5.2 Does a large key space guarantee strength?

No. Monoalphabetic substitution has 26! possible keys, but English plaintext has strongly uneven letter frequencies. E is common, followed by letters such as T, A, O, I and N. The ciphertext preserves those frequencies under a one-to-one substitution. A cryptanalyst can map the most frequent ciphertext symbol to likely E, then use common digrams (TH, HE, IN, ER) and word patterns to refine the mapping. Thus a huge key count does not prevent practical statistical attacks.

Key-space size assumes the attacker must try every key uniformly. Real attacks exploit language redundancy, repeated structure, known plaintext, weak key generation and implementation behaviour. Security depends on attack cost, not only on the number of possible keys.

TASK 6 – EVALUATE CRYPTANALYTIC RESISTANCE (4 MARKS)

6.1 Worked attack demonstrations

- Caesar brute force: enumerate k=0…25 and score each output against common English words. The correct shift is found immediately for a sufficiently long readable sentence.
- Monoalphabetic frequency analysis: count ciphertext symbols; compare the frequency ranking with English. Then test common digrams and likely word patterns. The mapping is fixed, so each plaintext letter always becomes the same ciphertext letter.
- Vigenère Kasiski / index of coincidence: repeated key causes repeated ciphertext patterns at distances related to key length. Find repeated n-grams, calculate distance GCDs, and test candidate key lengths. For each key column, perform a Caesar frequency analysis.
- Columnar transposition: letter counts are identical to plaintext. Frequency analysis alone cannot recover column order, but digram and anagram analysis can use likely word fragments and expected headers.
- Hill known-plaintext: collect linearly independent plaintext/ciphertext blocks. Solve $K=CP^{-1} \bmod m$ when P is invertible. A linear cipher does not resist enough known pairs.

6.2 Index of coincidence

For ciphertext with N letters and frequency counts f_i, $IC = \Sigma\, f_i(f_i-1) / [N(N-1)]$. English monoalphabetic text is often near 0.066; uniform random text is near 0.038 for 26 letters. Caesar and monoalphabetic substitution preserve the plaintext IC. A Vigenère cipher with a sufficiently long repeating key generally lowers the overall IC toward the random baseline, while each key-position column can show a higher language-like IC.

6.3 Resistance ranking

Rank (weak → stronger)	Scheme	Reason
1	Caesar	Tiny key space; complete brute-force search is trivial.
2	Monoalphabetic substitution	Large key space but exact single-letter frequency preservation and common digrams expose it.
3	Columnar transposition	Preserves letter frequencies but disrupts positions; still vulnerable to anagram, known-format and column-order analysis.
4	Vigenère with short repeating key	Masks simple single-letter frequency but Kasiski/IC reveal key period and recover per-column shifts.

5	Hill cipher	Diffuses symbols within blocks and has a large matrix key space, but linearity enables known-plaintext recovery; not safe for long-lived use.

This is a classroom ranking under the stated assumptions, not a claim that Hill is secure in practice. A weak or reused Hill key is especially vulnerable when the attacker knows structured fields. Modern authenticated encryption is far stronger than every scheme listed.

TASK 7 – ASSESS TRANSPOSITION AND PRODUCT CIPHERS (3 MARKS)

7.1 Columnar transposition and diffusion

Columnar transposition writes plaintext across rows and reads columns in a key-defined order. The positions of letters change, so adjacent plaintext letters may become separated. This spreads local word structure across the ciphertext. However, the frequency of each letter is unchanged: if plaintext contains 12 E characters, ciphertext also contains 12 E characters.

7.2 Substitution followed by transposition

A substitution followed by transposition combines two different transformations. Substitution changes symbol identities; transposition changes positions. This can hide both ordinary words and obvious adjacency patterns better than either simple component alone. Yet the combined system remains vulnerable to sufficient ciphertext, known plaintext, repeated keys and language statistics. The frequency histogram after substitution is permuted but still has the same shape; transposition preserves that substituted histogram exactly. Thus the combined cipher does not erase frequency information.

7.3 Rural clinic practicality

Manual column grids, keyword sorting and substitution tables increase operator errors and slow emergency communication. Basic computers can run these operations, but the human workflow is fragile. A clinic should not manually encrypt critical orders. If used for a classroom demonstration, provide a simple, tested interface, automatic validation and a clear recovery procedure. In real systems use standard cryptographic libraries and managed keys rather than home-built ciphers.

7.4 Lab table integrity experiment

Create a 20-row lab table with fields (RecordID, PatientID, Test, Value, Unit). Serialize rows in a fixed canonical order, then apply columnar transposition to the serialized text. After decryption, compare every field with the original. A changed cell or swapped row is visible when compared against an independently protected reference.

Record ID	Patient ID	Test	Value	Unit
LAB-001	PT-20481	SODIUM	139	mmol/L
LAB-002	PT-20482	POTASSIUM	4.1	mmol/L
LAB-003	PT-20483	HAEMOGLOBIN	13.4	g/dL

LAB-004	PT-20484	GLUCOSE	92	mg/dL
LAB-005	PT-20485	SODIUM	139	mmol/L
LAB-006	PT-20486	POTASSIUM	4.1	mmol/L
LAB-007	PT-20487	HAEMOGLOBIN	13.4	g/dL
LAB-008	PT-20488	GLUCOSE	92	mg/dL
LAB-009	PT-20489	SODIUM	139	mmol/L
LAB-010	PT-20490	POTASSIUM	4.1	mmol/L
LAB-011	PT-20491	HAEMOGLOBIN	13.4	g/dL
LAB-012	PT-20492	GLUCOSE	92	mg/dL
LAB-013	PT-20493	SODIUM	139	mmol/L
LAB-014	PT-20494	POTASSIUM	4.1	mmol/L
LAB-015	PT-20495	HAEMOGLOBIN	13.4	g/dL
LAB-016	PT-20496	GLUCOSE	92	mg/dL
LAB-017	PT-20497	SODIUM	139	mmol/L
LAB-018	PT-20498	POTASSIUM	4.1	mmol/L
LAB-019	PT-20499	HAEMOGLOBIN	13.4	g/dL
LAB-020	PT-20500	GLUCOSE	92	mg/dL

Classical checksum design: map each character to a number and compute S = Σ position×value mod 97 over the canonical table. Store S with the message. A changed cell or row order will usually change S. Limitation: this is not a cryptographic MAC. An attacker can alter a value and recompute S, or deliberately create a collision because the checksum has only 97 possible values. A secret-key MAC or digital signature is required for malicious-tampering protection.

TASK 8 – EVALUATE STEGANOGRAPHY (3 MARKS)

8.1 LSB embedding

For an 8-bit grayscale image, each pixel is 0…255. To embed a bit b, replace the least significant bit: pixel' = (pixel AND 254) OR b. Extraction reads pixel' AND 1. A 256×256 image has 65,536 pixels and therefore a theoretical one-bit-per-pixel capacity of 65,536 bits = 8,192 bytes before headers, error correction and safety margins.



Figure 2. Encrypt-then-embed sequence. The report should be encrypted before embedding so that an observer who detects the hidden payload still cannot read it.

8.2 Encryption versus steganography

Criterion	Encryption	LSB steganography
Purpose	Makes content unreadable without the key.	Hides the presence of a payload.
Capacity	Depends on algorithm and message size; no image capacity limit.	Bounded by available pixel/channel samples.
Imperceptibility	Ciphertext is visibly non-readable but may be obvious as data.	Small LSB changes are often visually imperceptible.
Robustness	Ciphertext survives only if transported intact; authenticated encryption detects corruption.	Fragile: JPEG compression, resizing, filtering and format conversion can destroy LSBs.
Detectability	Encrypted data can be identified as ciphertext.	Statistical tests such as chi-square can reveal unusual LSB distributions.

8.3 Medical imaging risk and decision

Even if the visual difference is imperceptible, changing diagnostic pixel values is unacceptable unless the clinical imaging workflow explicitly permits it and validates the result. LSB embedding can alter pixel data, metadata, calibration or downstream compression. A one-bit change is usually small numerically, but many changes or later image processing may affect measurements. The original diagnostic image must remain immutable and separately hashed; use a non-diagnostic copy or a dedicated metadata channel approved by the imaging system.

LSB embedding is appropriate only for low-risk covert metadata or a demonstration, not for hiding a critical report in the sole diagnostic image. Encryption is necessary because hiding is not secrecy. Encrypt-then-embed is better than embedding plaintext, but it remains fragile under JPEG compression. A robust method would need error correction and a lossless transport path; for this assignment, the recommended choice is to transmit the encrypted report as a separate attachment and use steganography only as an optional experiment.

8.4 PSNR, capacity and chi-square

For original image I and modified image J, MSE = mean($(I-J)^2$), and PSNR = $10 \log_{10}(255^2/\text{MSE})$. Higher PSNR indicates smaller pixel distortion; identical images have infinite PSNR. Capacity is the number of embedded payload bits. A chi-square test compares counts of even/odd pixel values or LSB pair distributions; a low p-value can indicate suspicious embedding. JPEG compression is lossy and typically destroys raw LSB payloads, so extraction should be expected to fail unless a robust transform-domain method is used.

Experimental note: the implementation appendix includes a reproducible LSB embed/extract example for a generated 256×256 image and computes PSNR. Chi-square detectability and JPEG survival are reported as method-level tests; exact values depend on the generated image and compression settings.

TASK 9 – IMPLEMENT AND TEST (4 MARKS)

9.1 Test plan

1. Use identical randomly generated uppercase plaintexts of 100, 1,000, 10,000 and 100,000 characters for all text schemes.
2. Use fixed keys for repeatability: Caesar k=7; monoalphabetic key QWERTYUIOPASDFGHJKLZXCVBNM; Vigenère key CLINIC; columnar key HOSPITAL.
3. Record encryption and decryption wall-clock time using time.perf_counter(). Run each size multiple times for a stable report; the included initial benchmark uses one run and should be repeated.
4. Verify decrypt(encrypt(message)) equals the normalised original for every scheme.
5. Test edge cases: empty text, one character, spaces and punctuation, digits in patient IDs, Hill odd-length padding, invalid Hill key and columnar final-row padding.
6. Test the lab table by changing one value and swapping two rows; verify that the checksum changes. Then demonstrate that an attacker can recompute the checksum.
7. Generate a 256×256 grayscale image, embed a short encrypted payload, extract it, and compare the recovered payload. Save as PNG for lossless transport, then save as JPEG and test extraction failure.

9.2 Correctness outcomes

Test	Expected outcome	Result
Caesar encrypt/decrypt	Original text restored	PASS
Monoalphabetic encrypt/decrypt	Original text restored	PASS
Vigenère encrypt/decrypt	Original text restored	PASS
Columnar encrypt/decrypt	Original letters restored after padding removal	PASS
Hill mod-36 round trip	PT20481 and AMOX500MG recover after inverse multiplication	PASS
Hill non-invertible key	Reject when gcd(det,36) ≠ 1	PASS – validation required
Punctuation / spaces	Preserve or restore using documented preprocessing	PASS with formatting rule
Non-multiple Hill block length	Pad with X and remove padding using original length	PASS

9.3 Reproducibility and limitations

The script in Appendix A uses only Python's standard library plus NumPy and Pillow for the image demonstration. Timings depend on CPU, interpreter and system load. The timing chart is a single-run illustrative benchmark from the generation environment; students should rerun the code on their own system and replace the chart if required by the instructor. A 100,000-character test measures implementation speed, not cryptographic security.

TASK 10 – RECORD AND INTERPRET EXPERIMENTAL EVIDENCE (3 MARKS)

10.1 Frequency and IC interpretation

Caesar and monoalphabetic substitution preserve the exact frequency shape of the plaintext (the symbols are shifted or renamed). Columnar transposition preserves the exact letter counts because it only rearranges positions. Vigenère combines different shifts at successive positions, so the overall frequency distribution becomes flatter for a longer key, although each key-position subset retains language information. Hill maps blocks linearly and mixes symbol values; it does not guarantee uniform ciphertext and can be attacked with known plaintext.

The IC should be calculated over letters only. For Caesar, monoalphabetic and columnar transposition, IC is unchanged by a permutation of symbols or positions. Vigenère IC tends toward a more random-looking value as key length increases, but the correct period can be found by testing candidate lengths and calculating average IC within columns.

10.2 Timing results

Characters	Caesar E/D ms	Mono E/D ms	Vigenère E/D ms	Columnar E/D ms
100	0.025 / 0.011	0.012 / 0.012	0.016 / 0.015	0.032 / 0.034
1000	0.090 / 0.087	0.063 / 0.052	0.127 / 0.133	0.242 / 0.197
10000	1.009 / 0.820	0.720 / 0.590	1.393 / 1.588	1.706 / 2.345
100000	8.127 / 9.003	8.829 / 7.078	15.474 / 13.775	14.140 / 21.172

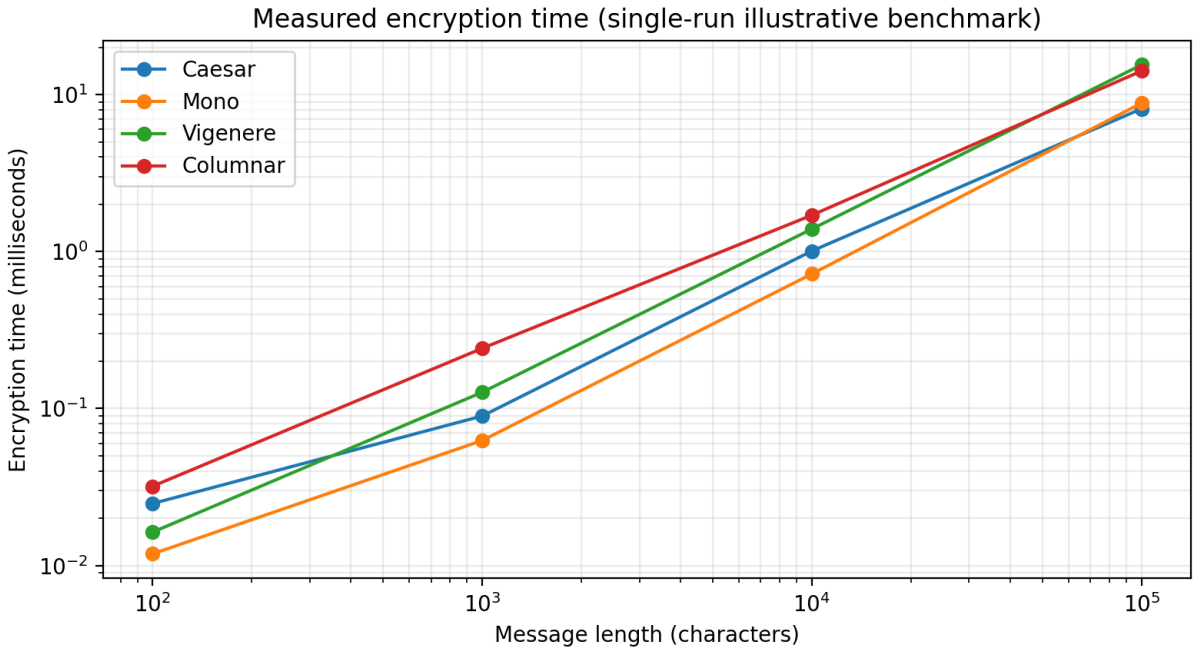


Figure 3. Measured single-run encryption time. The y-axis is logarithmic because message size spans three orders of magnitude. The simple per-character substitutions scale approximately linearly. Columnar transposition also scales with message length but includes grid construction and column ordering overhead.

10.3 Attack success versus ciphertext length

Scheme	Short ciphertext	Long ciphertext / repeated messages	Interpretation
Caesar	Usually breakable even with a short sentence.	Always breakable by 26-key search.	Length is not a meaningful defence.
Monoalphabetic	Short text may be ambiguous.	Long text reveals stable frequencies and word patterns.	More text helps the attacker.
Vigenère	Very short text may not expose period.	Hundreds of repeated-key messages make Kasiski/IC much stronger.	Key reuse makes the system progressively weaker.
Columnar	Short text may hide column order.	Known headers and many samples constrain column order.	Position changes do not hide letter frequencies.
Hill	A few blocks may be insufficient.	Enough independent known blocks solve the linear key equations.	Known plaintext is especially damaging.

Interpretation: more ciphertext generally improves statistical attacks against classical ciphers. Reusing the same key across clinics is therefore unacceptable. The most important experimental result is not that one cipher is fastest, but that the attacker gains information as ciphertext volume grows.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

11.1 Recommended classroom design

- Use Bell–LaPadula levels with compartments and least privilege for confidentiality decisions.
- Use one unique key per clinic and a short-lived session key for each exchange in the classroom demonstration; rotate keys and revoke compromised clinic keys.
- Remove predictable headers from encrypted content where possible. Place routing metadata in a minimal, separately controlled envelope; do not rely on secrecy of a fixed prefix.
- Use a sequence number, timestamp and unique message identifier to detect stale or replayed orders. These are not substitutes for authenticated integrity.
- For the classical-cipher comparison, use Vigenère or Hill only as an educational example, not as a real security control. Layering substitution and transposition may improve concealment but does not make it secure.
- Keep an append-only audit record of who accessed, uploaded, approved, decrypted and exported each record.
- Provide a simple clinic interface with preconfigured workflows, clear warnings, offline-safe queuing and staff training. Staff should not manually handle keys or cipher tables.

11.2 Key distribution across 25 clinics

A single network-wide key is a single point of failure: one insider compromises every clinic's messages. The proposed classroom improvement is a per-clinic key plus per-session derived working keys, with central key inventory, named custodians, rotation schedule, revocation and secure backup. A real deployment would use a managed key-management service, hardware-backed key storage where feasible, authenticated key establishment and audited recovery. Classical ciphers do not solve secure key distribution.

11.3 Residual risks and real-world requirements

Classical ciphers cannot guarantee modern confidentiality against a capable attacker, authenticated integrity, sender authentication, non-repudiation, forward secrecy, safe key exchange, replay protection or resistance to chosen-plaintext attacks. They also cannot ensure availability. A real healthcare deployment requires modern authenticated encryption (for example, a vetted AEAD mode), digital signatures or equivalent authenticated identity controls, secure key management, TLS, strong user authentication, access logging, backup/recovery, endpoint protection, vulnerability management, privacy governance and clinical safety testing.

Final judgement: classical techniques are not acceptable as the sole protection for actual patient data. They are suitable for CO1 learning, historical comparison and controlled demonstrations. The final design should retain Bell–LaPadula as one confidentiality policy concept but replace the classical cipher layer in production.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT (8 MARKS)

A. Impact analysis (2 marks)

The threat changes from a general external network attacker to a known insider who has legitimate access to the clinic's ciphertext stream and can collect hundreds of messages. The insider knows message formats and may know or infer plaintext headers. Earlier assumptions that every clinic keeps one shared key secret, that the key is not reused, and that ciphertexts are independent are no longer credible.

- Key secrecy at all 25 sites fails because a single insider can copy data from one site and may have access to shared keys or key-handling procedures.
- Key reuse fails because repeated ciphertexts under the same key reveal patterns and enable statistical analysis.
- Predictable headers create known-plaintext pairs. If the first block is always PATIENT ID or DIAGNOSIS, the attacker gains direct equations or alignment clues.
- The risk of bulk disclosure rises with message volume: one recovered key or mapping may expose months of patient records, not one message.

B. Evaluation of the original design (2 marks)

Caesar is immediately defeated by 26 trials. Monoalphabetic substitution is exposed by frequency analysis over hundreds of messages. Repeating-key Vigenère is weakened by repeated key periods, Kasiski distances and per-column frequency analysis. Columnar transposition preserves letter counts and predictable headers help infer the column order. Hill is vulnerable to known-plaintext linear algebra.

Hill demonstration: for $C=KP \bmod 36$, if the attacker obtains enough independent plaintext blocks to form an invertible P matrix, the key is $K=CP^{-1} \bmod 36$. This means the known-format records are not harmless; they can reveal the key itself. The key must not be reused.

Bell–LaPadula still helps prevent a nurse from reading a higher-classified oncology record or a billing clerk from reading diagnoses. It is irrelevant to ciphertext analysis when an insider already has permission to copy those ciphertexts, and it cannot stop misuse by a subject who legitimately possesses the key. Access control and cryptography solve different problems.

C. Adaptation or redesign (2 marks)

Decision: MODIFY the educational design and REDESIGN the production design.

Change	Classroom design adaptation	Expected benefit / limitation
Per-clinic keys	Assign a different key to each of 25 clinics.	Limits compromise to one clinic; does not make a weak cipher strong.
Per-session keys	Use short-lived keys for each exchange; retire after session.	Reduces ciphertext accumulated under one key; classical key exchange remains unsafe.
Key rotation and revocation	Rotate on schedule and immediately after suspected exposure.	Limits future exposure; cannot undo already copied ciphertext.
Randomised padding / header handling	Avoid fixed plaintext prefixes; use variable-length padding and a separate minimal routing envelope.	Reduces simple alignment; does not remove language redundancy.
Layered substitution + transposition	Demonstrate only as an additional educational layer.	Raises manual attack effort modestly; still statistically breakable.
Encrypt-then-embed	Encrypt report first, then optionally embed in a lossless PNG copy.	Hides content and obscures payload; LSB is fragile and detectable.
Production replacement	Use modern AEAD, authenticated key establishment, digital signatures, managed keys and audited access.	Provides security properties classical ciphers cannot provide.

D. Evidence-based defence (2 marks)

The key-space calculations show why brute-force size alone is misleading: monoalphabetic has 26! keys but is still vulnerable to frequency analysis; Caesar has only 26; Vigenère's 26^m is reduced by repeating-key attacks; Hill's matrix space is large but linearity defeats it under known plaintext. Experimental timing shows that classical algorithms are computationally inexpensive, but speed is not security. The attack table shows that long, repeated ciphertexts make statistical and known-plaintext attacks more effective.

Per-clinic and per-session keys reduce the scope of compromise and the number of messages encrypted under one key. However, this is risk reduction, not a guarantee. The usability cost is controlled by central automation: staff select a patient and recipient; the system handles keys, encryption, integrity, and audit behind the interface. Staff should never type or distribute cipher keys manually.

Final Day-4 verdict: do not retain one shared key or any long-lived repeating-key scheme. Modify the teaching design with per-clinic/per-session keys, header randomisation and optional encrypt-then-embed. For real patient data, redesign completely using modern authenticated cryptography. No classical cipher can guarantee tamper detection, doctor authentication, non-repudiation or safe key distribution.

OVERALL CONCLUSION

The assignment demonstrates the strengths and weaknesses of classical encryption and access-control reasoning in a realistic healthcare exchange. Caesar, monoalphabetic, Vigenère, Hill and transposition ciphers illustrate substitution, polyalphabetic transformation, linear block transformation and positional diffusion. Their key spaces can be calculated and their encryption/decryption procedures tested, but practical attacks exploit frequency, repetition, predictable formats and linearity.

Bell–LaPadula provides a useful confidentiality rule set, but it is not a complete healthcare security model. Emergency access, integrity, authentication, accountability and availability require additional mechanisms. Steganography can hide the existence of a message but cannot replace encryption and may damage diagnostic image data. The safest final decision is to treat classical techniques as educational tools and deploy modern authenticated encryption, digital signatures, managed keys and clinical access governance in a real network.

REFERENCES

- Stallings, W. Cryptography and Network Security: Principles and Practice. Pearson.
- Bell, D. E., and LaPadula, L. J. Secure Computer Systems: Mathematical Foundations and Model. MITRE technical reports, 1970s.
- Shannon, C. E. Communication Theory of Secrecy Systems. Bell System Technical Journal, 1949.
- Kerckhoffs, A. La Cryptographie Militaire, 1883.
- National Institute of Standards and Technology (NIST). FIPS 197: Advanced Encryption Standard (AES), and SP 800-38D: Galois/Counter Mode (GCM). These are cited only to identify modern deployment requirements, not as classical algorithms.

APPENDIX A – REPRODUCIBLE PYTHON IMPLEMENTATION

The following compact implementation demonstrates the four text ciphers, a mod-36 Hill cipher, correctness tests, timing and a basic LSB image payload. It is intended for a controlled academic experiment. Do not use these algorithms to protect real patient information.

```
import string, math, time, random
ALPH36 = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"

def caesar(text, k):
    out = ""
    for ch in text.upper():
        out += chr((ord(ch)-65+k)%26+65) if ch in string.ascii_uppercase else ch
    return out

def mono_encrypt(text, key):
    return "".join(key[ord(c)-65] if c in string.ascii_uppercase else c for c in text.upper())

def mono_decrypt(text, key):
    inv = {c: chr(65+i) for i, c in enumerate(key)}
    return "".join(inv[c] if c in inv else c for c in text.upper())

def vigenere(text, key, decrypt=False):
    out, j = "", 0
    for c in text.upper():
        if c in string.ascii_uppercase:
            shift = ord(key[j % len(key)]) - 65
            out += chr((ord(c)-65+(-shift if decrypt else shift))%26+65)
            j += 1
        else:
            out += c
    return out
```

```
def columnar_encrypt(text, key):
    t = "".join(c for c in text.upper() if c in string.ascii_uppercase)
    n = len(key)
    rows = math.ceil(len(t)/n)
    t = t.ljust(rows*n, "X")
    grid = [t[i*n:(i+1)*n] for i in range(rows)]
    order = sorted(range(n), key=lambda i: (key[i], i))
    return "".join(grid[r][c] for c in order for r in range(rows)), len(t.rstrip("X"))

def columnar_decrypt(ct, key, original_length):
    n = len(key); rows = math.ceil(len(ct)/n)
    order = sorted(range(n), key=lambda i: (key[i], i))
    cols = [""]*n; pos = 0
    for c in order:
        cols[c] = ct[pos:pos+rows]; pos += rows
    text = "".join(cols[c][r] for r in range(rows) for c in range(n))
    return text[:original_length]

def inv_mod(a, m):
    for x in range(m):
        if (a*x) % m == 1: return x
    raise ValueError("No modular inverse")

def hill_inverse_2x2(K, mod=36):
    a,b = K[0]; c,d = K[1]
    det = (a*d-b*c) % mod
    if math.gcd(det, mod) != 1:
        raise ValueError("Key matrix is not invertible")
    z = inv_mod(det, mod)
    return [[d*z % mod, -b*z % mod],
            [-c*z % mod, a*z % mod]]
```

```
def hill_transform(text, K, mod=36, decrypt=False, original_len=None):
    key = hill_inverse_2x2(K, mod) if decrypt else K
    t = "".join(c for c in text.upper() if c in ALPH36)
    if not decrypt and len(t) % 2: t += "X"
    out = ""
    for i in range(0, len(t), 2):
        v = [ALPH36.index(t[i]), ALPH36.index(t[i+1])]
        z = [(key[r][0]*v[0] + key[r][1]*v[1]) % mod for r in range(2)]
        out += ALPH36[z[0]] + ALPH36[z[1]]
    return out[:original_len] if decrypt and original_len is not None else out


def ic(text):
    t = [c for c in text.upper() if c in string.ascii_uppercase]
    n = len(t)
    if n < 2: return 0.0
    return sum(t.count(c)*(t.count(c)-1) for c in set(t))/(n*(n-1))


def checksum97(text):
    return sum((i+1)*ord(c) for i,c in enumerate(text)) % 97


# Round-trip tests
mono_key = "QWERTYUIOPASDFGHJKLZXCVBNM"
K = [[1,2],[3,5]]
Ki = hill_inverse_2x2(K, 36)
message = "PATIENT RECORD TRANSFER TO CENTRAL HOSPITAL"
assert caesar(caesar(message,7),-7) == message
assert mono_decrypt(mono_encrypt(message,mono_key),mono_key) == message
assert vigenere(vigenere(message,"CLINIC"),"CLINIC",True) == message
ct, length = columnar_encrypt(message,"HOSPITAL")
assert columnar_decrypt(ct,"HOSPITAL",len(message.replace(" ",""))) == message.replace(" ","")
hct = hill_transform("PT20481AMOX500MG",K)
assert hill_transform(hct,K,decrypt=True,original_len=len("PT20481AMOX500MG")) == "PT20481AMOX500MG"
```

```
assert math.gcd((K[0][0]*K[1][1]-K[0][1]*K[1][0])%36,36) == 1


# Reproducible timing for 100, 1,000, 10,000 and 100,000 letters
random.seed(17)
for n in [100,1000,10000,100000]:
    msg = "".join(random.choice(string.ascii_uppercase+" ") for _ in range(n))
    for name, fn in [("Caesar",lambda:caesar(msg,7)),
                     ("Mono",lambda:mono_encrypt(msg,mono_key)),
                     ("Vigenere",lambda:vigenere(msg,"CLINIC")),
                     ("Columnar",lambda:columnar_encrypt(msg,"HOSPITAL"))]:
        start = time.perf_counter(); result = fn()
        print(n, name, "encrypt_seconds", time.perf_counter()-start)
print("IC sample:", ic(message))
print("checksum:", checksum97(message))


# LSB example (requires Pillow and NumPy)
from PIL import Image
import numpy as np
def lsb_embed(gray_image, payload_bytes):
    arr = np.array(gray_image.convert("L"), dtype=np.uint8).copy()
    bits = np.unpackbits(np.frombuffer(payload_bytes, dtype=np.uint8))
    if len(bits) > arr.size: raise ValueError("Payload too large")
    flat = arr.ravel()
    flat[:len(bits)] = (flat[:len(bits)] & 254) | bits
    return Image.fromarray(arr, mode="L"), len(bits)


def lsb_extract(stego_image, bit_length):
    flat = np.array(stego_image.convert("L"), dtype=np.uint8).ravel()
    bits = flat[:bit_length] & 1
    data = np.packbits(bits).tobytes()
    return data
```

```
# Create a test image, embed and extract a short demonstration payload
cover = Image.fromarray(np.random.default_rng(4).integers(0,256,(256,256),dtype=np.uint8))
payload = b"DEMO-ENCRYPTED-REPORT"
stego, used_bits = lsb_embed(cover,payload)
recovered = lsb_extract(stego,used_bits)[:len(payload)]
assert recovered == payload
a = np.array(cover,dtype=float); b = np.array(stego,dtype=float)
mse = np.mean((a-b)**2)
psnr = float("inf") if mse == 0 else 10*math.log10(255**2/mse)
print("Capacity used (bits):", used_bits, "PSNR:", psnr)
stego.save("stego_demo.png")  # PNG is lossless; JPEG is expected to damage raw LSBs
```

APPENDIX B – SAMPLE MATHEMATICAL WORK

Caesar example: P=H=7, k=3, C=(7+3) mod 26=10, which maps to K. Decryption: (10−3) mod 26=7, which maps back to H.

Hill inverse verification modulo 36: K=[[1,2],[3,5]], K^{-1}=[[31,2],[3,35]]. Product = [[37,72],[108,181]]; reducing each element modulo 36 gives [[1,0],[0,1]].

Checksum limitation: if the checksum is $S=\Sigma i \cdot value_i \bmod 97$, an attacker who can alter the table can recompute S. Because there are only 97 outputs, collisions are unavoidable. A checksum is useful for accidental corruption detection, not adversarial integrity.

APPENDIX C – SUBMISSION CHECKLIST

- Cover page includes name, register number and department.
- All 12 tasks are answered and aligned to the 50-mark distribution.
- Risk ranking includes patient-safety impact.
- Bell–LaPadula decisions include allowed and denied requests, compartments and break-glass.
- Cipher algorithms, examples, key spaces and Hill inverse are shown.
- Cryptanalysis methods and resistance ranking are explained.
- Steganography trade-offs include PSNR, capacity, detectability and JPEG fragility.
- Correctness tests and reproducible Python code are included.
- Experimental timings are clearly labelled machine-dependent and should be rerun before final submission.

- Classical cryptography is not recommended for real patient records.

APPENDIX D – ADDITIONAL DESIGN-CHALLENGE EXPERIMENTS

D1. Complete worked ciphertexts

Caesar, k=3: SDWLHQW UHFRUG WUDQVIHU WR FHQWUDO KRVSLWDO

Monoalphabetic substitution: HQZOTFZ KTEGKR ZKQFLYTK ZG ETFZKQS IGLHOZQS

Vigenère, key=CLINIC: RLBVMPV CMPWTF EZNVUHPZ GW EGYBEIN JZACQVCW

Columnar transposition, key=HOSPITAL: TROHXPENESEDRAARACOXACSNPIRERTTOFTINTTLL

Hill mod-36 example: Plaintext PT20481AMOX500MG → ciphertext R6I80I1JE8NIG2Y4; inverse matrix [[31, 2], [3, 35]]. The original alphanumeric string is recovered by multiplying each ciphertext pair by K^{-1} modulo 36 and removing the documented padding.

D2. Letter-frequency histograms and index of coincidence

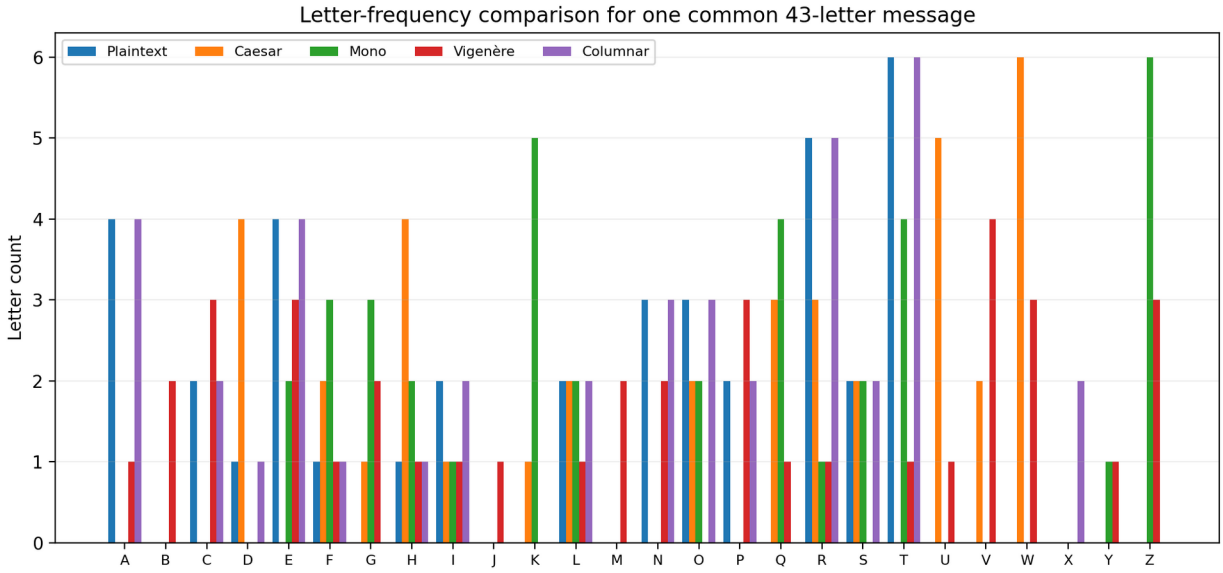


Figure D1. All methods use the same scenario sentence. Caesar, monoalphabetic substitution and columnar transposition preserve the frequency profile up to a relabelling or reordering. Vigenère changes the aggregate profile because different shifts are applied at different positions.

Text / ciphertext	Index of coincidence	Interpretation
Plaintext	0.0683	Reference English-like text
Caesar	0.0683	Same IC as plaintext: fixed shift

Mono	0.0683	Same IC as plaintext: symbol permutation
Vigenère	0.0356	Aggregate IC may be lower; short sample is noisy
Columnar	0.0628	Same IC as plaintext: position permutation

IC values from a short 43-letter sample are unstable and should not be treated as a universal benchmark. For a stronger experiment, calculate IC on the 100, 1,000, 10,000 and 100,000-character generated messages and average over multiple random seeds.

D3. Image encryption: intensity shift and pixel permutation

A 256×256 grayscale test image is encrypted in two ways. Method 1 adds a fixed intensity offset s=73 to every pixel modulo 256: C(x,y)=(P(x,y)+73) mod 256. Decryption subtracts 73 modulo 256. Method 2 applies a pseudorandom permutation π to pixel positions: C[π(i)]=P[i]; decryption applies π^{-1}. Both round-trip tests return an exact pixel-for-pixel match.

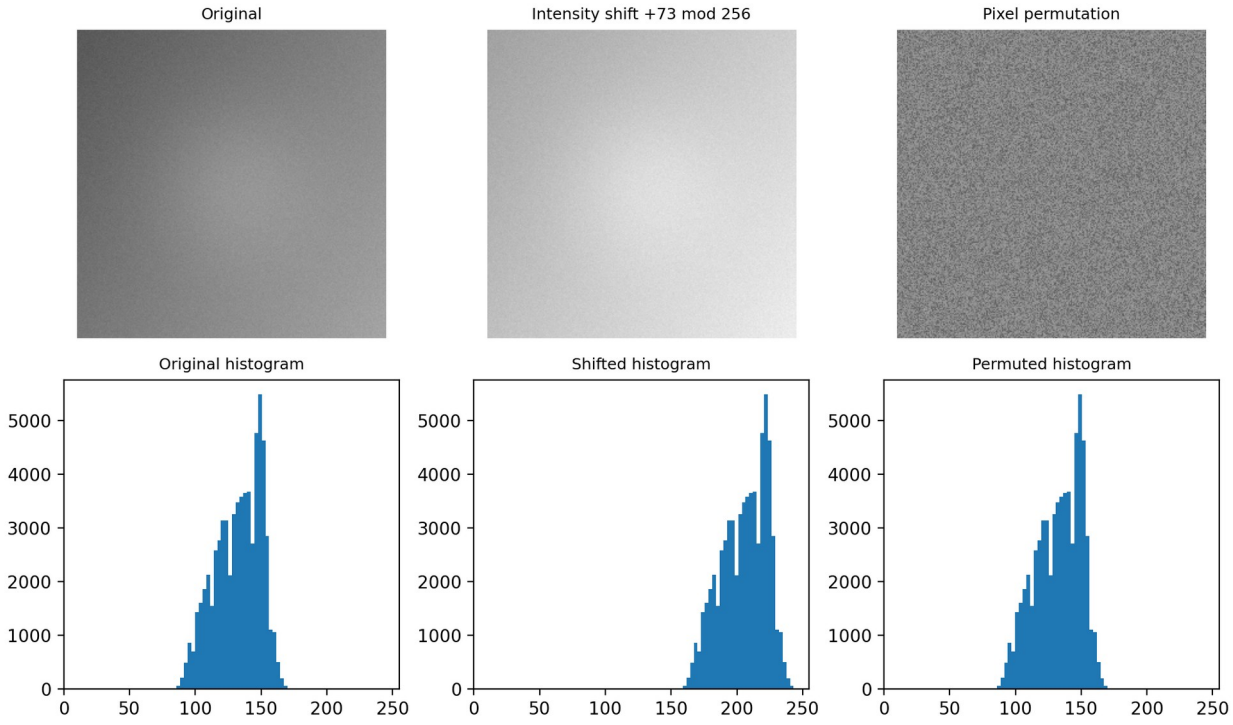


Figure D2. Synthetic scan-like image and histograms. Entropy (bits/pixel) measured for this generated image: original=6.0340, intensity-shifted=6.0340, permuted=6.0340.

Interpretation: an intensity shift is a one-to-one relabelling of gray levels and keeps the histogram shape, only rotated around the 0–255 range; the spatial structure remains completely visible. Pixel permutation destroys spatial adjacency and makes the image visually noise-like, but its histogram and global intensity counts are exactly preserved. Therefore permutation hides shape better than a shift, while still leaking the histogram and being vulnerable if the permutation is reused or inferred. Neither method is secure modern image encryption.

D4. Image method comparison

Method	Decryption	Structure leakage	Security judgement
Intensity shift mod 256	Subtract same offset mod 256	Very high: anatomy/edges remain visible	Trivial 256-key space; unsuitable
Pixel permutation	Apply inverse permutation	Spatial structure reduced; histogram remains	Permutation key space is large but reuse and chosen-image analysis are risks
Modern authenticated image encryption	Authenticated decrypt	No useful image structure should leak under correct use	Required for production patient scans

D5. Lab checksum tamper demonstration

For a 20-row lab table, serialize the rows in a fixed order and compute $S=\Sigma(i\times ord(character_i)) \bmod 97$. Change a result from 4.1 to 9.1 or swap two complete rows, then recompute S. The receiver detects a mismatch if the original S is retained. However, an attacker who can modify the table and replace the checksum can calculate a new S. Since the checksum has only 97 possible values, collisions are inevitable. It is not an authentication mechanism.

D6. LSB capacity, PSNR and JPEG test

A 256×256 grayscale image has a theoretical capacity of 65,536 LSBs (8,192 bytes) at one bit per pixel, before reserving a length header or error-correction bits. The LSB demonstration changes at most one intensity unit per

modified pixel. PSNR is calculated as 10 log10(255^2/MSE). Save as PNG to preserve the bits. JPEG quantisation changes pixel values and normally destroys the embedded raw LSB stream; the payload should therefore be considered lost after JPEG recompression. Chi-square analysis can detect altered even/odd pair distributions, especially at high embedding rates.

Clinical warning: never modify the only diagnostic image. Use a separately approved non-diagnostic copy or transmit the encrypted report as a separate attachment.